

REPUBLIQUE DU CAMEROUN

PAIX -TRAVAIL-PATRIE

INSTITUT AFRICAINE
D'INFORMATIQUE

BP: 13 719 Yaoundé (Cameroun)

Tel: (237) 22 72 99 57 / (237) 22
72 99 58



REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

AFRICAN INSTITUTE OF
COMPUTERS SCIENCES

BP: 13 719 Yaoundé (Cameroun)

Tel: (237) 22 72 99 57 / (237) 22 72
99 58

WEB PROGRAMMING TWO

THEME :

FEEDBACK PLATFORM APP

Designated and implemented by a student in AICS Cameroon in view of
**UNDERSTANDING PROJECT CARRIED OUT FROM THE 21st MARCH, 2026 TO
30TH MARCH, 2026**

Series: Software Engineering

PRESENTED BY:

FOUCHE HAMIDA

CLASS: BA2A

SUPERVISED AND COORDINATED BY:

Mr. TCHOUA

ACADEMIC YEAR :

2025/2026

Contents

TABLE OF FIGURES	3
1. PROJECT IDENTIFICATION.....	4
2. INTRODUCTION	4
3. EXISTING SYSTEM	4
4. PROBLEMATIC	5
5. LIMITATIONS OF EXISTING FEATURES	5
6. PROPOSED SOLUTION	6
7. GENERAL SYSTEM DESCRIPTION	6
8. ACTORS IDENTIFICATION.....	7
9. FUNCTIONAL REQUIREMENT	7
10. USE CASE ANALYSIS	8
TEXTUAL DESCRIPTION OF THE USE CASE.....	10
11. USER INTERACTION SCENARIOS	11
12. BUSINESS RULES	12
13. VALIDATION AND RESPONSES.....	13
14. NAVIGATION LOGIC	14
15. DATABASE DESIGN.....	15
16. API DESIGN	16
17. SYSTEM ARCHITECTURE	16
18. NON- FUNCTIONAL REQUIREMENT	17
19. TESTING.....	17
20. DEPLOYMENT	18
21. DELIVERABLES.....	20
22. SCREENSHOTS.....	21
23. CONCLUSION.....	29

TABLE OF FIGURES

Figure 1: usecase diagram.....	9
Figure 2: entity relation diagram.....	15
Figure 3:home page.....	21
Figure 4:feedback page	22
Figure 5: submit feedback page	24
Figure 6: Ratings	26
Figure 7: Analytics.....	27
Figure 8: profile page.....	27
Figure 9:sign up page.....	28
Figure 10: login page	28

1. PROJECT IDENTIFICATION

- Project Title: Feedback Platform
- Student Name: FOUCHE HAMIDA
- Technologies Used: HTML, CSS, JavaScript, PHP, MySQL
- Target Users: Students, customers, organizations, administrators
- Project Category: Web Application (User Feedback Management System)
- Link to the application online **13.60.27.180**

2. INTRODUCTION

The Feedback Platform is a web-based application designed to allow users to submit feedback, rate services, and share suggestions or complaints.

In many organizations, feedback collection is either manual or poorly structured, making it difficult to analyse and act upon. This project aims to digitize and centralize feedback collection, making it easier to manage, analyse, and improve services.

The main objective is to provide a user-friendly system that enables efficient communication between users and administrators while supporting decision-making through analytics.

Educational institutions rely heavily on feedback to improve teaching quality, administrative services, and overall student experience. Currently, most feedback collection relies on annual surveys, suggestion boxes, or informal channels methods that are often inefficient, untimely, and difficult to analyse systematically.

3. EXISTING SYSTEM

Annual Paper Surveys: Distributed at semester end, collected manually, data entry performed by administrative staff

1. **Email Suggestions:** Informal emails to department heads with no tracking mechanism
2. **Physical Suggestion Boxes:** Anonymous but lack follow-up capabilities
3. **Verbal Feedback:** During office hours or meetings, often undocumented
4. **Learning Management System (LMS) Modules:** Basic rating systems without detailed analytics

4. PROBLEMATIC

The existing system has several issues:

- Time-consuming manual processes
- No centralized storage
- Poor accessibility
- Lack of real-time analysis
- No structured rating system

Issue	Impact
Time-Consuming Processes	Manual data collection and entry take weeks; feedback is stale when analysed
Lack of Automation	No automatic routing of feedback to relevant departments; everything goes through central administration
Poor Accessibility	Paper forms require physical presence; email systems exclude students without institutional accounts
Inefficiency	Low response rates due to survey fatigue; no incentive for participation
No Real-Time Insights	Decision-makers cannot identify emerging issues until formal reports are compiled
Limited Transparency	Submitters never know if feedback was received, read, or acted upon

5. LIMITATIONS OF EXISTING FEATURES

Technical Weaknesses

- **No Data Tracking:** Paper forms are easily lost; emails are deleted; no audit trail exists
- **Security Concerns:** Physical suggestion boxes offer no authentication; anonymous submissions cannot be verified as legitimate
- **Scalability Issues:** Manual processes become unmanageable as institution size increases
- **Error-Prone:** Data entry mistakes during transcription; illegible handwriting
- **No Integration:** Feedback data isolated from student information systems or learning analytics

Functional Limitations

- **No Categorization:** All feedback treated equally regardless of urgency or type
- **Limited Analytics:** Basic counting only; no trend analysis or sentiment detection
- **No Mobile Support:** Paper and desktop-only systems exclude mobile-first users
- **Poor User Experience:** Long, boring forms discourage completion

6. PROPOSED SOLUTION

The proposed web application transforms feedback management through digital automation, intelligent routing, and comprehensive analytics. It creates a closed-loop system where feedback is submitted, tracked, responded to, and analysed continuously

- **Real-Time Submission:** Instant feedback capture with mobile-responsive interface
- **Automated Routing:** Feedback automatically categorized and assigned to relevant departments
- **Status Tracking:** Submitters can track progress from "Received" to "Under Review" to "Resolved"
- **Analytics Dashboard:** Visual charts showing feedback trends, response times, and satisfaction scores
- **Gamification:** Points and badges encourage quality feedback and active participation
- **Anonymous Option:** Verified anonymity protects users while preventing abuse

Core Value Proposition

The platform bridges the communication gap between stakeholders by making feedback a continuous conversation rather than a periodic event. It transforms feedback from a compliance exercise into a strategic improvement tool.

7. GENERAL SYSTEM DESCRIPTION

Workflow:

- ❖ User logs into the system
- ❖ Navigates to dashboard
- ❖ Submits feedback with rating
- ❖ Feedback is stored in database
- ❖ Admin views and analyses feedback

8. ACTORS IDENTIFICATION

User	Administrator
Register/Login	View all feedback
Submit feedback	Analyse data
View own feedback	Manage users
Rate services	Monitor system

9. FUNCTIONAL REQUIREMENT

Authentication & Authorization

- User registration with email verification
- Secure login with password hashing (bcrypt)
- Password reset functionality
- Session management with timeout
- Role-based access control (RBAC)

Feedback Management

- Submit feedback with title, description, category, priority
- Support for anonymous submissions (optional)
- File attachment support (images, documents)
- Edit/delete own feedback within 24 hours

Ratings System

- Rate feedback quality (1-5 stars)
- Rate resolution satisfaction
- View personal rating statistics
- Leaderboard for top contributors

Profile & Settings

- Update personal information and avatar
- Notification preferences (email, in-app)
- Privacy settings (anonymous default)
- View activity history and achievements

10. USE CASE ANALYSIS

A use case diagram represents the interaction between the user and the offline video playing system.

Main Use Cases:

- Login
- Submit Feedback
- View Feedback
- Rate Service
- View Analytics
- Each use case defines interaction between user and system.

USE CASE DIAGRAM

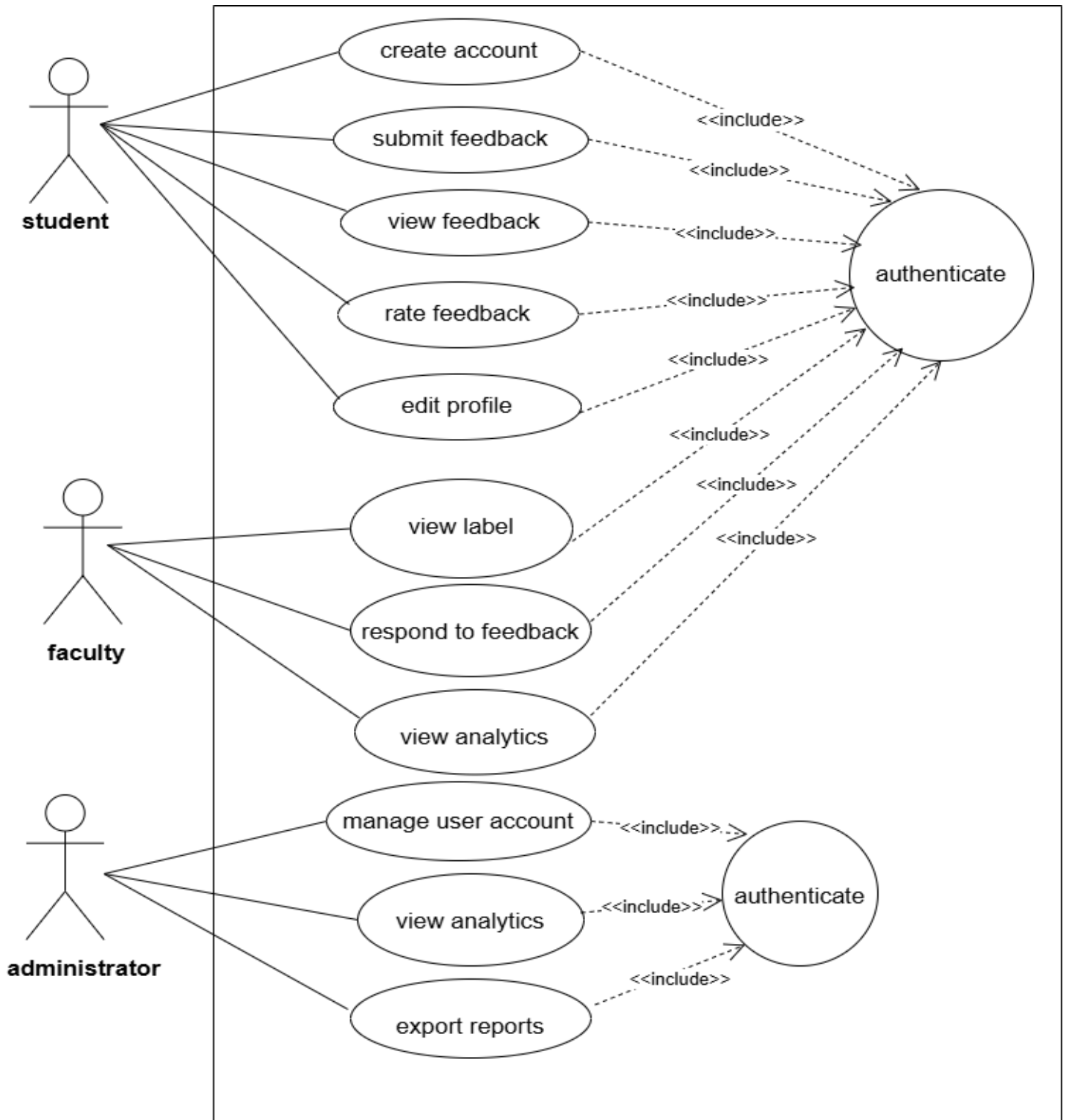


Figure 1: usecase diagram

TEXTUAL DESCRIPTION OF THE USE CASE

Use Case 1: Submit Feedback

Actor: student

Precondition: User is authenticated or has an account

Date: 27/02/2026

Stakeholder: Amida

nominal scenario:

1. User clicks "Submit Feedback" button
2. System displays feedback form
3. User selects category (Bug, Feature, Improvement, General)
4. User enters title and detailed description
5. User selects priority level (Low, Medium, High, Critical)
6. User chooses anonymous option (optional)
7. User submits form
8. System validates input and saves feedback
9. System assigns unique tracking ID
10. System notifies relevant department
11. User receives confirmation with tracking ID

Alternative Flow:

3a. User uploads screenshot: System validates file type and size

7a. Validation fails: System displays error messages, returns to form

Post condition of success: the feedback is sent successively

Post condition of failure: the feedback is not submitted.

Use Case 2: Respond to Feedback

Actor: faculty

Precondition: User has permission to view specific feedback

Date: 27/03/2026

Stakeholder: Anita

nominal scenario:

1. User views feedback list filtered by department
2. User selects specific feedback item
3. System displays feedback details and history
4. User clicks "Respond" button
5. User enters response text and selects new status
6. User submits response
7. System updates feedback status
8. System notifies original submitter (if not anonymous)

Written by a student at AICS Cameroon Technological Center of Excellence 2025-2026

9. Response is added to feedback thread

Post condition of success: the user send the responds respectively

11. USER INTERACTION SCENARIOS

Scenario 1: Normal Flow - Student Submitting Course Feedback

Context: Sarah, a second-year student, wants to suggest improvements for her Database Management course.

1. Sarah logs into the Feedback Platform using her student credentials
2. Dashboard shows her recent activity and quick action buttons
3. She clicks "Submit Feedback" (prominent purple button)
4. Form loads with her profile pre-filled
5. She selects "Course Feedback" category
6. She chooses "Improvement" as the type
7. She enters title: "More Practical SQL Exercises Needed"
8. She writes detailed description explaining that theory is strong but hands-on practice is limited
9. She sets priority to "Medium"
10. She keeps "Anonymous" unchecked (willing to be identified)
11. She clicks Submit
12. System shows success message with confetti animation
13. Feedback appears in her "My Submissions" list with status "Submitted"
14. She receives email confirmation with tracking

Scenario 2: Error Scenario - Invalid Login Attempt

Context: John tries to access the platform but forgot his password.

1. John enters email: john.doe@university.edu
2. John enters incorrect password: "password123"

3. System validates credentials against database
4. Validation fails
5. System displays error: "Invalid email or password" (generic for security)
6. Login attempt is logged (IP address, timestamp)
7. After 3 failed attempts, CAPTCHA appears
8. John clicks "Forgot Password"
9. System sends password reset link to email
10. John checks email and clicks secure link
11. System validates token and shows password reset form
12. John enters new password meeting complexity requirements
13. System updates password hash and invalidates old sessions
14. Success message: "Password updated successfully"
15. Redirect to login page

12. BUSINESS RULES

Fields	Rules	Messages
Email	Valid format, institutional domain	Please enter a valid email
Password d	Min 8 characters, 1 uppercase, 1 number, 1 special character.	Password must meet complexity requirements
Feedback title	10-100 characters, no profanity	Title must be 10-100 characters
Description	Min 50 characters, max 2000	Description is too short or long
Category	Must select from predefined list	Please select a category
File upload	Max 5MB, JPG/PNG/PDF only	Invalid file type or size

13. VALIDATION AND RESPONSES

Validation Mechanisms

Client-Side Validation (JavaScript):

- Real-time email format checking
- Password strength indicator
- Character counters for text fields
- File type validation before upload

Server-Side Validation (PHP):

- Sanitize all inputs (strip_tags, htmlspecialchars)
- Prepared statements for database queries
- CSRF token validation on all forms
- Rate limiting on API endpoints

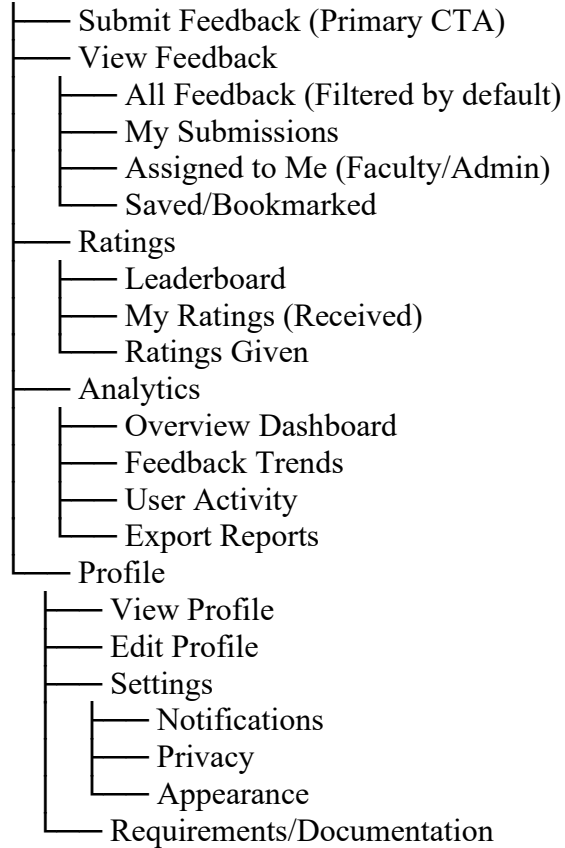
Success Responses

- Create Operations: HTTP 201 with created resource and success message
- Update Operations: HTTP 200 with updated resource and toast notification
- Delete Operations: HTTP 200 with confirmation and undo option (5 seconds)
- Read Operations: HTTP 200 with requested data and pagination metadata

14. NAVIGATION LOGIC

Navigation Structure:

Home/Dashboard



15. DATABASE DESIGN

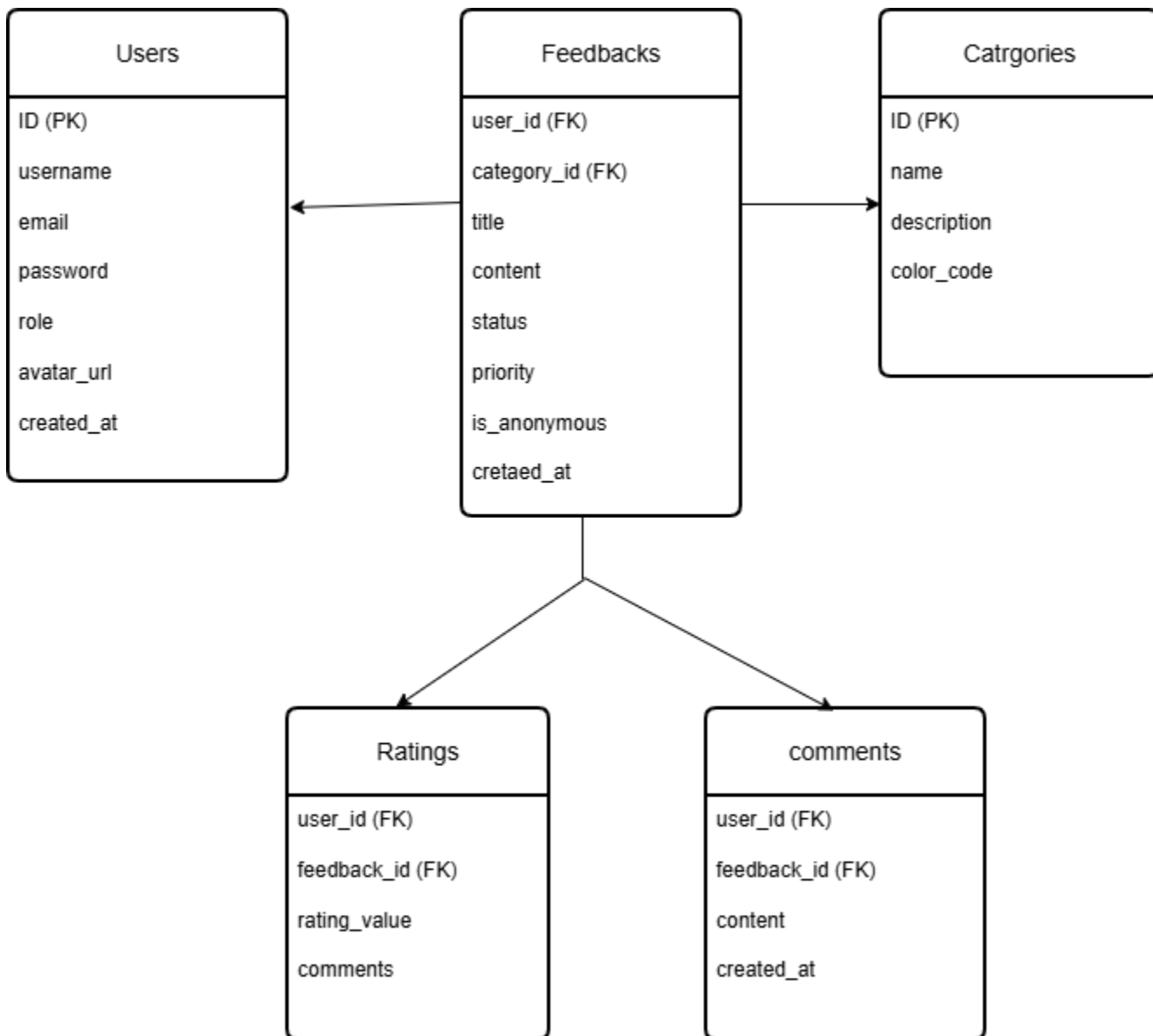


Figure 2: entity relation diagram

16. API DESIGN

Authentication Endpoints

Method	Endpoints	Description	Request	Responds
POST	/api/auth/register	Create new account	{name, email, password, role}	{user, token}
POST	/api/auth/login	Authenticate user	{email, password}	{message}
POST	/api/auth/logout	End session	Authorization: Bearer token	{message}
POST	/api/auth/refresh	Refresh token	Authorization: Bearer token	{token}
POST	/api/auth/forgot-password	Request reset	{email}	{message}
POST	/api/auth/reset-password	Set new password	{token, new_password}	{message}

17. SYSTEM ARCHITECTURE

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (Vanilla or Vue.js)	User interface, interactivity
Styling	Tailwind CSS or Custom CSS	Responsive design, animations
Backend	PHP 8.2+	Server-side logic, API
Framework	Custom MVC or Laravel	Application structure
Database	MySQL 8.0	Data persistence
Web Server	Apache or Nginx	HTTP requests, URL rewriting

Caching	Redis (optional)	Session storage, query caching
Email	PHPMailer or SendGrid API	Notifications
Storage	Local filesystem or AWS S3	File uploads

18. NON- FUNCTIONAL REQUIREMENT

Performance Requirements

- Page Load Time: < 2 seconds for initial load, < 500ms for API responses
- Concurrent Users: Support 500 simultaneous users
- Database Queries: < 100ms for standard queries, < 500ms for complex analytics
- File Upload: Support files up to 10MB with progress indication

Security Requirements

- Authentication: bcrypt password hashing (cost factor 12)
- Authorization: Role-based access control on all endpoints
- Data Protection: HTTPS/TLS 1.3 for all communications
- Input Sanitization: XSS prevention, SQL injection protection via prepared statements

19. TESTING

Authentication Module:

- Valid login redirects to dashboard
- Invalid password shows generic error
- Password reset email sent successfully
- Session expires after inactivity

Feedback Module:

- Valid feedback submission creates database record
- Anonymous submission hides user identity
- File upload validates type and size
- Edit permission expires after 24 hours
- Status transitions follow workflow rules

Analytics Module:

- Dashboard loads within 2 seconds
- Date range filters affect all charts
- Export generates valid CSV file
- Permission controls restrict data access

Manual Testing Checklist

- Cross-browser compatibility (Chrome, Firefox, Safari, Edge)
- Mobile responsiveness (iPhone, Android, Tablet)
- Accessibility (Screen reader navigation, color contrast)
- Edge cases (Empty states, Maximum length inputs, Special characters)
- Security (SQL injection attempts)

20. DEPLOYMENT

1. Launch Instance

Navigate to AWS Console → Services → EC2 → Launch instance. State the instance name and purpose. Describe the AMI choice (Amazon Linux 2023 for AWS optimization, Ubuntu for flexibility, or Windows for .NET apps). Justify the instance type selection based on CPU/memory requirements and budget.

2. Configure Key Pair

Create a new RSA key pair in .pem format. Emphasize that this is the only chance to download the private key—AWS does not store it. Describe setting file permissions to 400 to prevent unauthorized access.

3. Network Settings

Select default VPC and public subnet. Enable auto-assign public IP for internet accessibility. Define the security group as a virtual firewall with specific inbound rules: SSH restricted to your IP only (security best practice), HTTP/HTTPS open to all for web traffic, and application ports limited to specific sources.

4. Storage Configuration

Choose gp3 SSD for price-performance balance. Specify size (8-30 GB). State whether encryption is enabled for data protection compliance.

5. Launch and Verify

Click launch and monitor the instance state transition from pending to running. Confirm both system and instance status checks pass (2/2). This verifies the underlying hardware and network configuration are functional.

6. Connect to Instance

Use SSH with the private key file to establish secure terminal access. Mention the default username varies by AMI (ec2-user for Amazon Linux, ubuntu for Ubuntu). Confirm successful authentication and shell access.

7. Environment Setup

Update operating system packages to latest security patches. Install required runtime environments (Node.js, Java, Python, etc.) and dependencies specific to your application stack.

8. Application Deployment

Transfer application code via SCP, Git clone, or S3 download. Extract files to appropriate directory (/opt/ or /var/www/). Install application dependencies (npm install, pip install, etc.). Configure environment variables for database connections, API keys, and secrets.

9. Service Configuration

Create system service file to run application as background service. This enables auto-start on boot, automatic restart on crashes, and proper logging. Set service to run under dedicated non-root user for security isolation.

10. Production Hardening (Optional)

Deploy Application Load Balancer for traffic distribution and SSL termination using AWS Certificate Manager. Configure Auto Scaling Group for high availability across multiple availability zones. Implement CloudWatch for metrics and logging. Set up automated EBS snapshots for backup.

21. DELIVERABLES

Source Code

- Complete PHP application source code in ZIP or Git repository
- Directory structure following PSR-4 autoloading standards
- Comprehensive inline code comments
- README.md with installation instructions

Database

- SQL dump file with schema and seed data (database.sql)
- Migration scripts for version control
- Entity Relationship Diagram (ERD) in PDF format

Documentation

- User Manual (PDF): Step-by-step guide for students, faculty, and administrators
- Technical Documentation: API reference, system architecture diagrams
- This Project Report (PDF)

Deployment

- Live application URL: **13.60.27.180**
- Admin credentials for evaluation (provided separately)
- Demo video walkthrough (5-10 minutes) showing all features

22. SCREENSHOTS

1. Home page

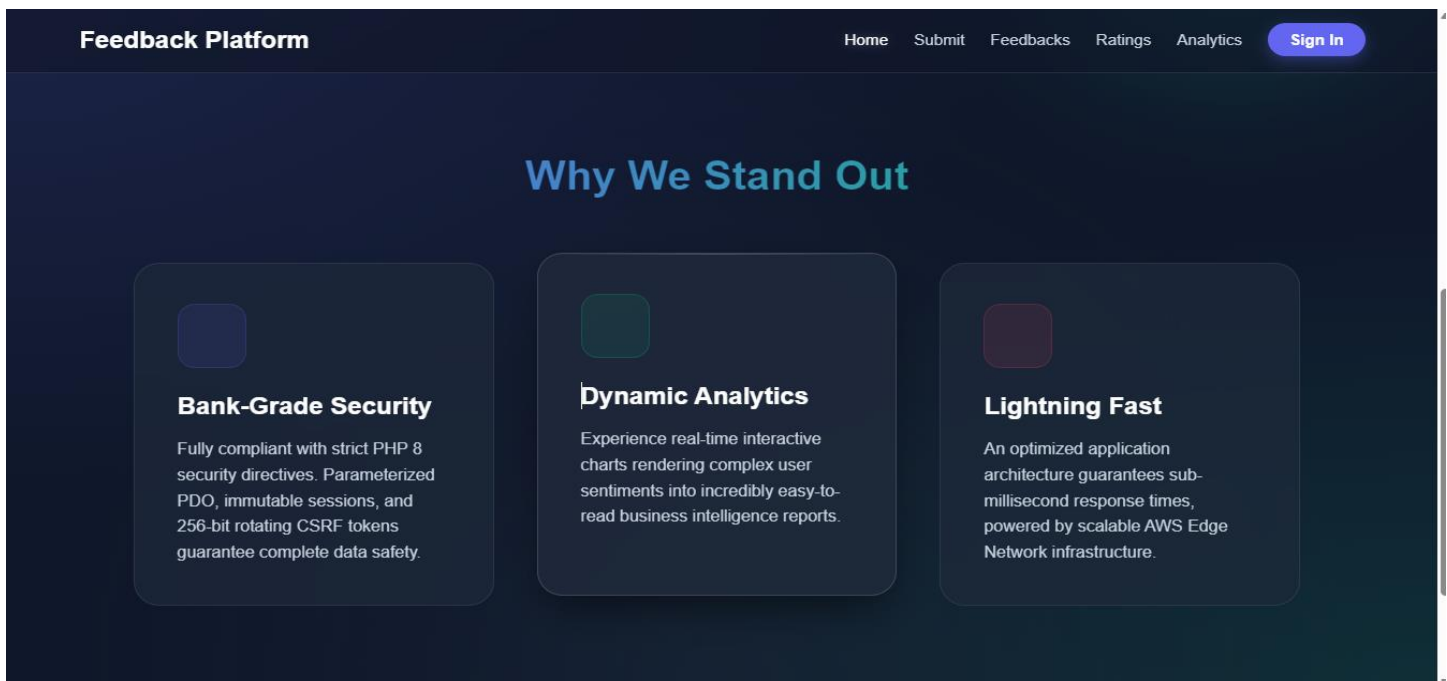
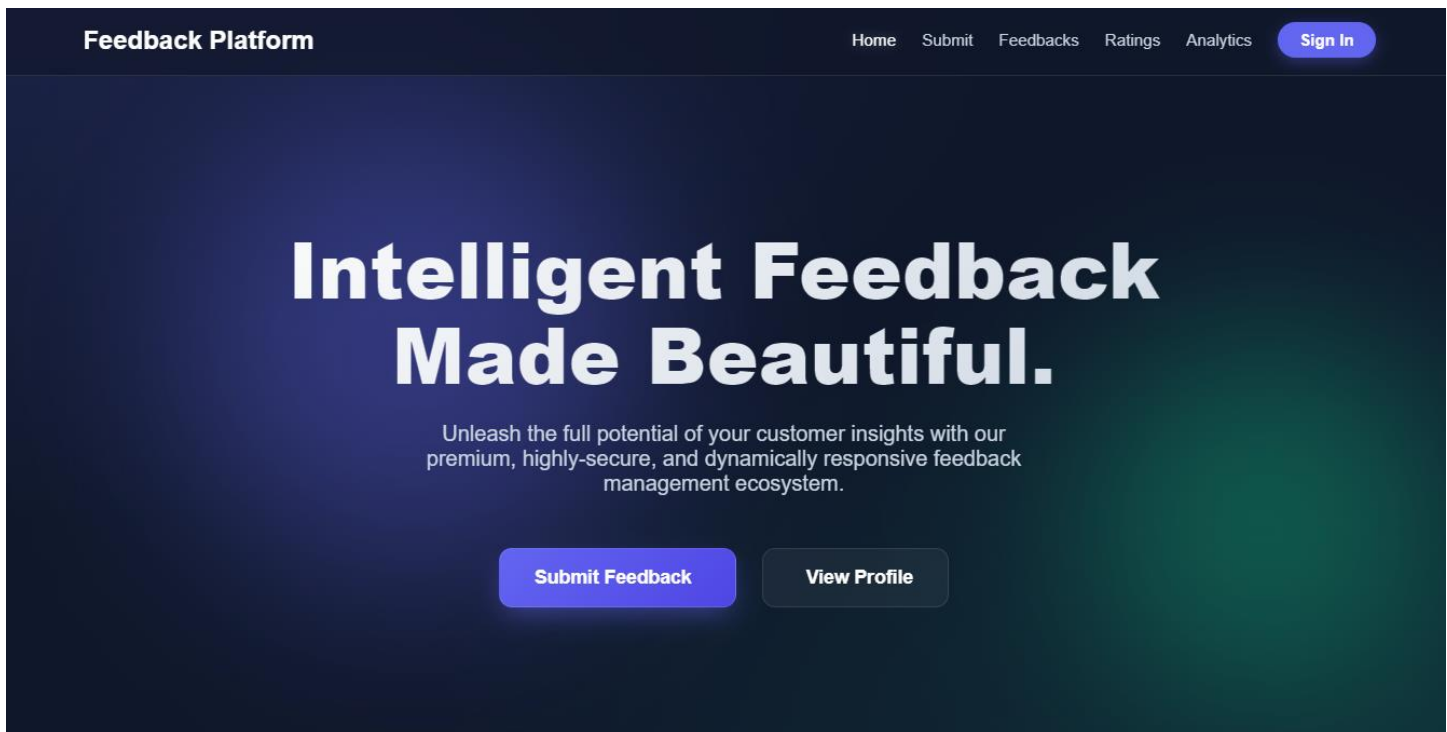


Figure 3:home page

2. Feedback

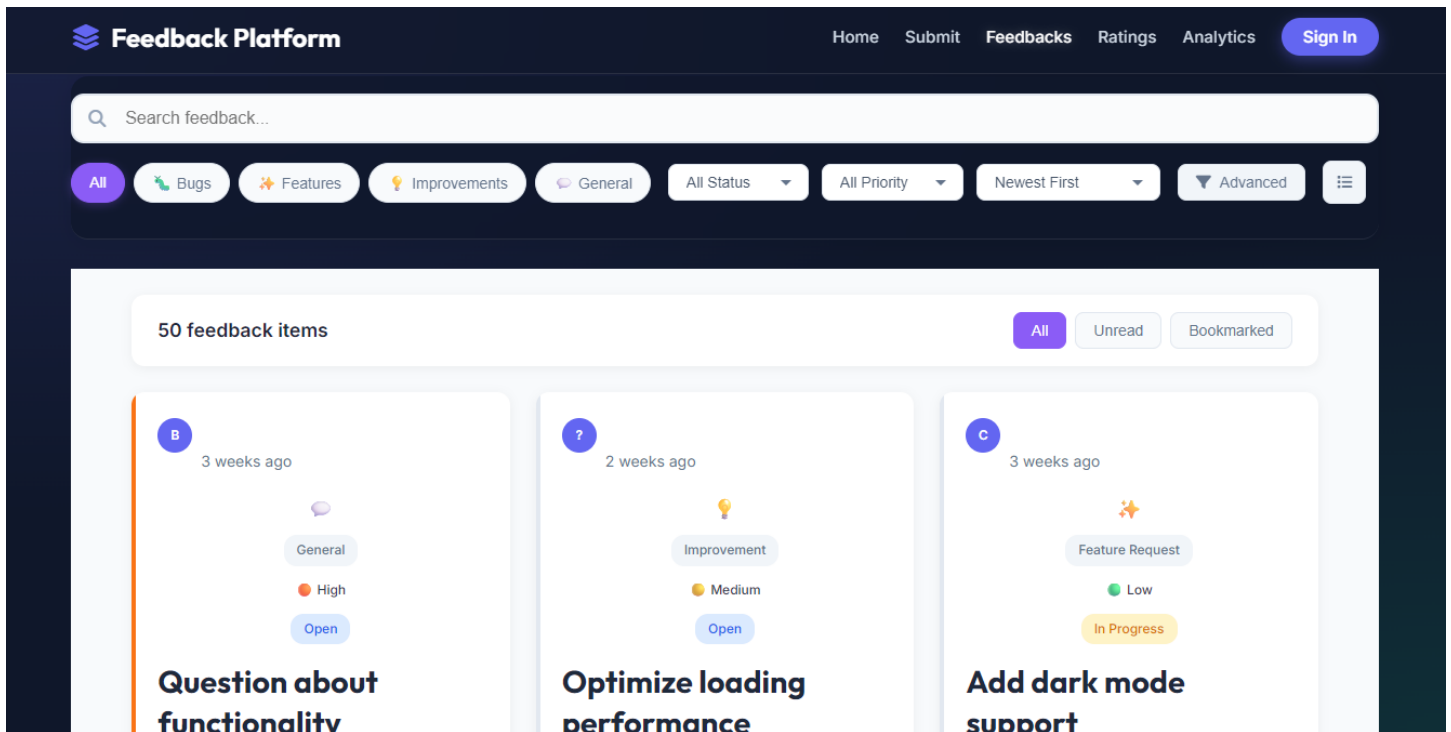
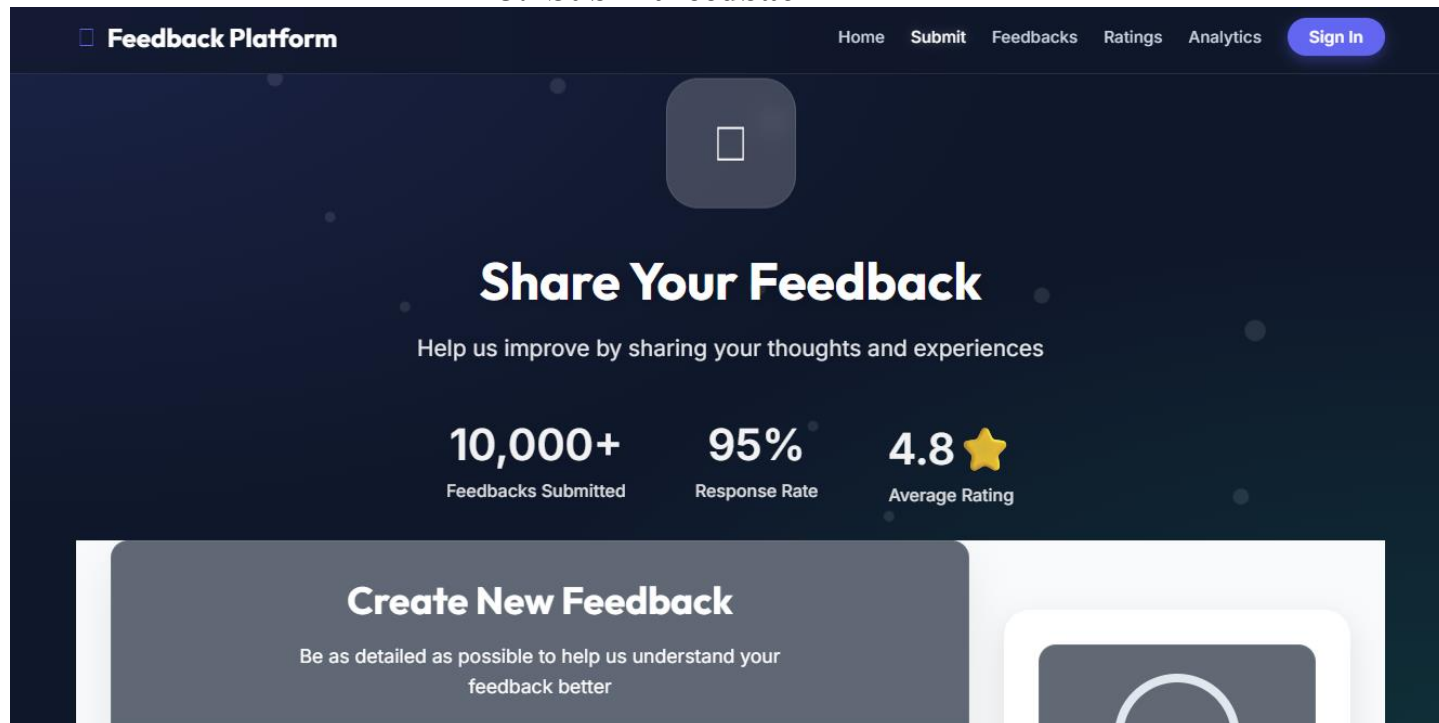


Figure 4: feedback page

3. Submit feedback




Feedback Platform

[Home](#)
[Submit](#)
[Feedbacks](#)
[Ratings](#)
[Analytics](#)
[Sign In](#)

Create New Feedback

Be as detailed as possible to help us understand your feedback better

H Feedback Title

0/100

What type of feedback is this?



Bug Report
Report an issue or problem



Feature Request
Suggest a new feature



Improvement
Enhance existing functionality

0

Feedback Quality Score

Complete more sections to improve your score

5

Points you'll earn

Base submission	+5
-----------------	----


Feedback Platform

[Home](#)
[Submit](#)
[Feedbacks](#)
[Ratings](#)
[Analytics](#)
[Sign In](#)



Bug Report
Report an issue or problem



Feature Request
Suggest a new feature



Improvement
Enhance existing functionality



General
Other feedback or comments

How important is this feedback?

5

Points you'll earn

Base submission	+5
Detailed description	+3
Attachments included	+2

Great feedback includes:

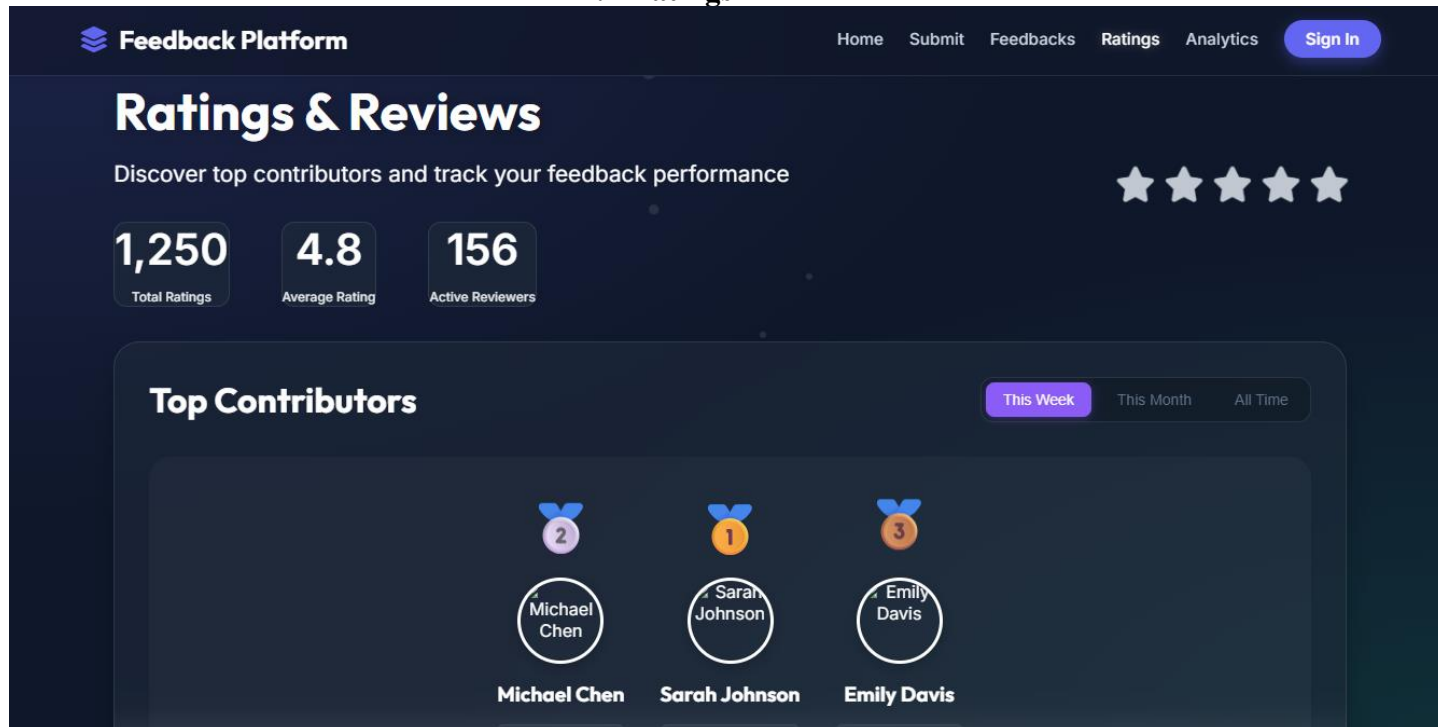
☐ Clear, descriptive title

☐ Appropriate category selection

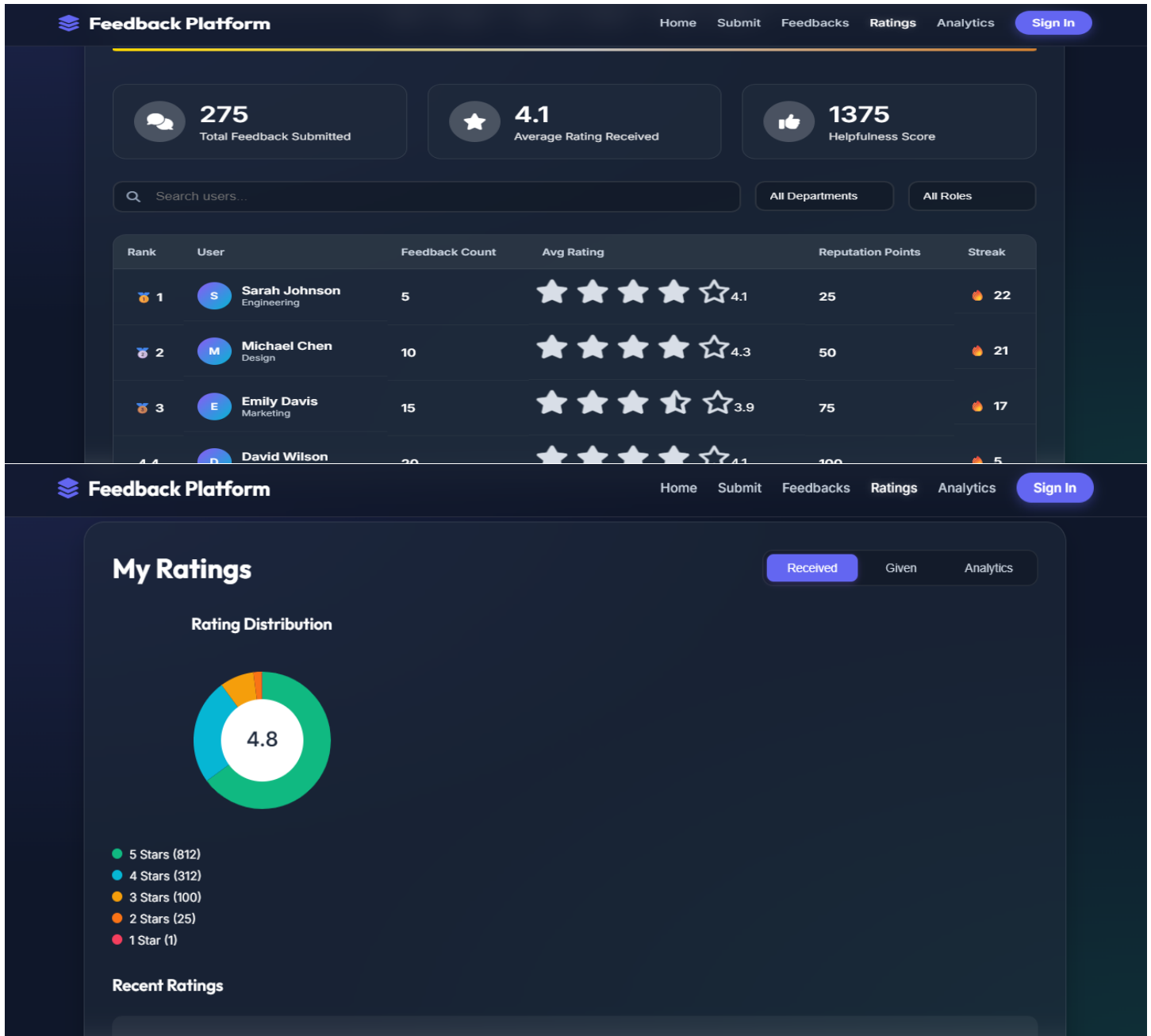
☐ Detailed

Figure 5: submit feedback page

4. Ratings



FEEDBACK PLATFORM APP



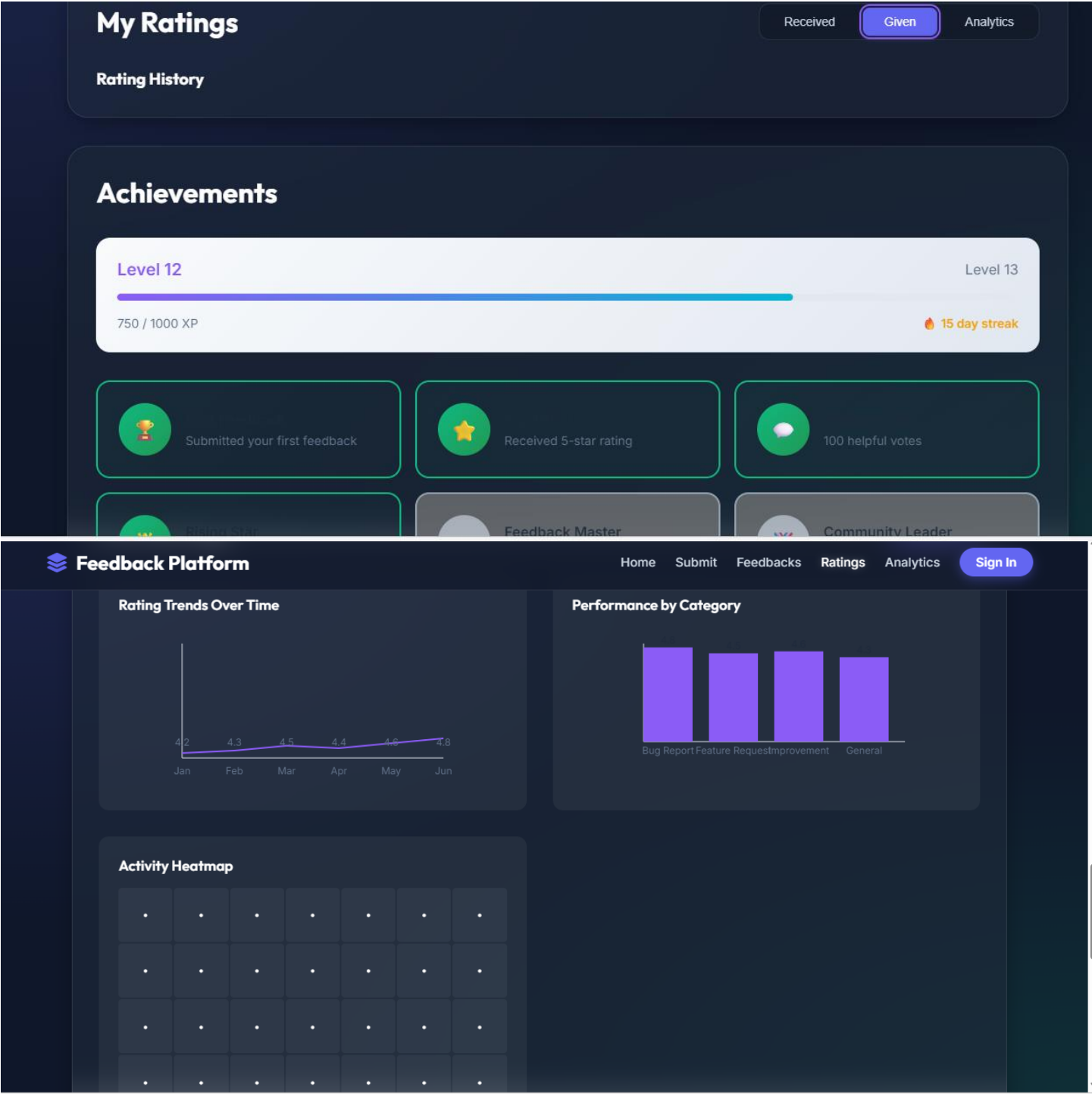


Figure 6: Ratings

5. Analytics

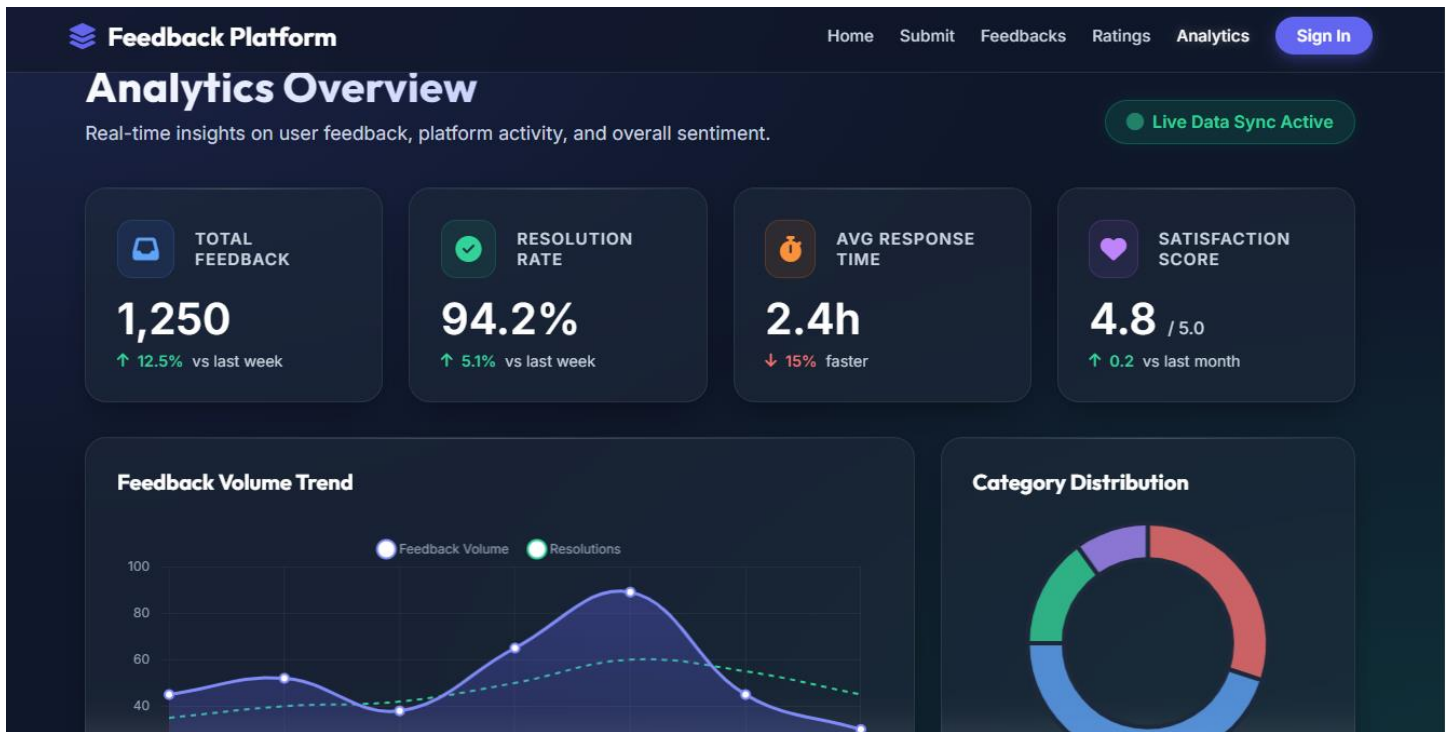


Figure 7: Analytics

6. Profile

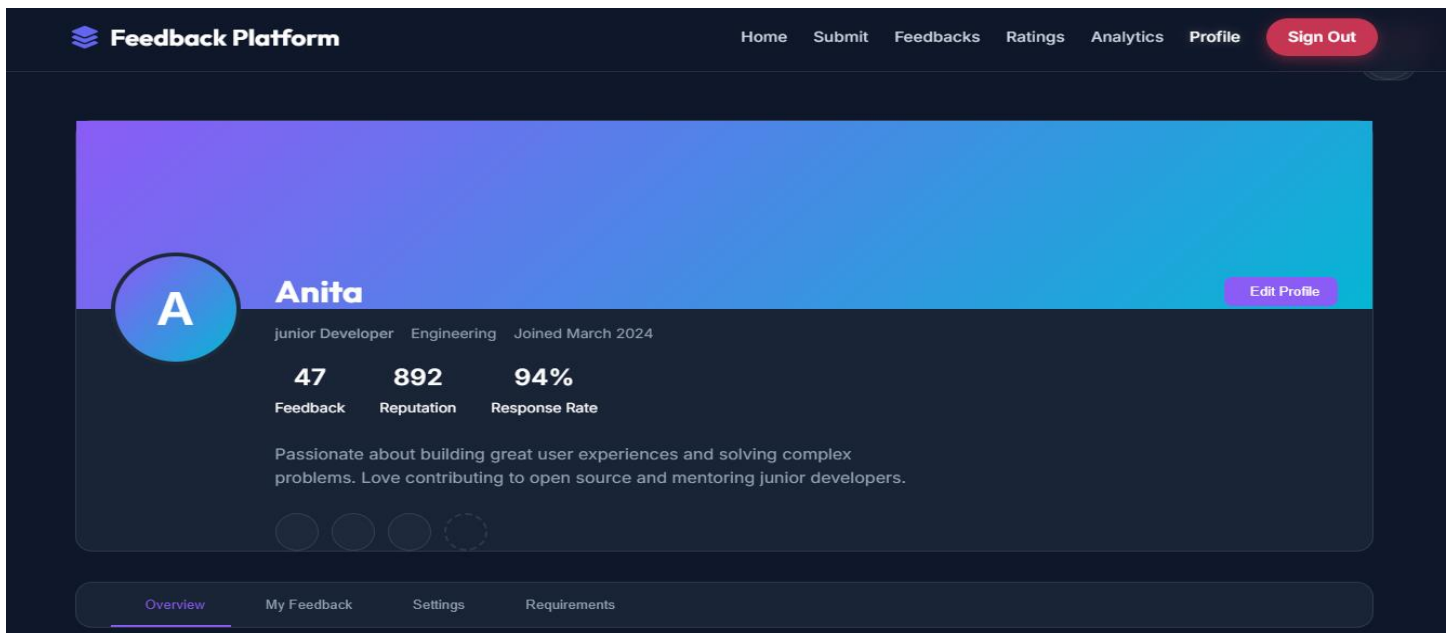


Figure 8: profile page

7. Sign in

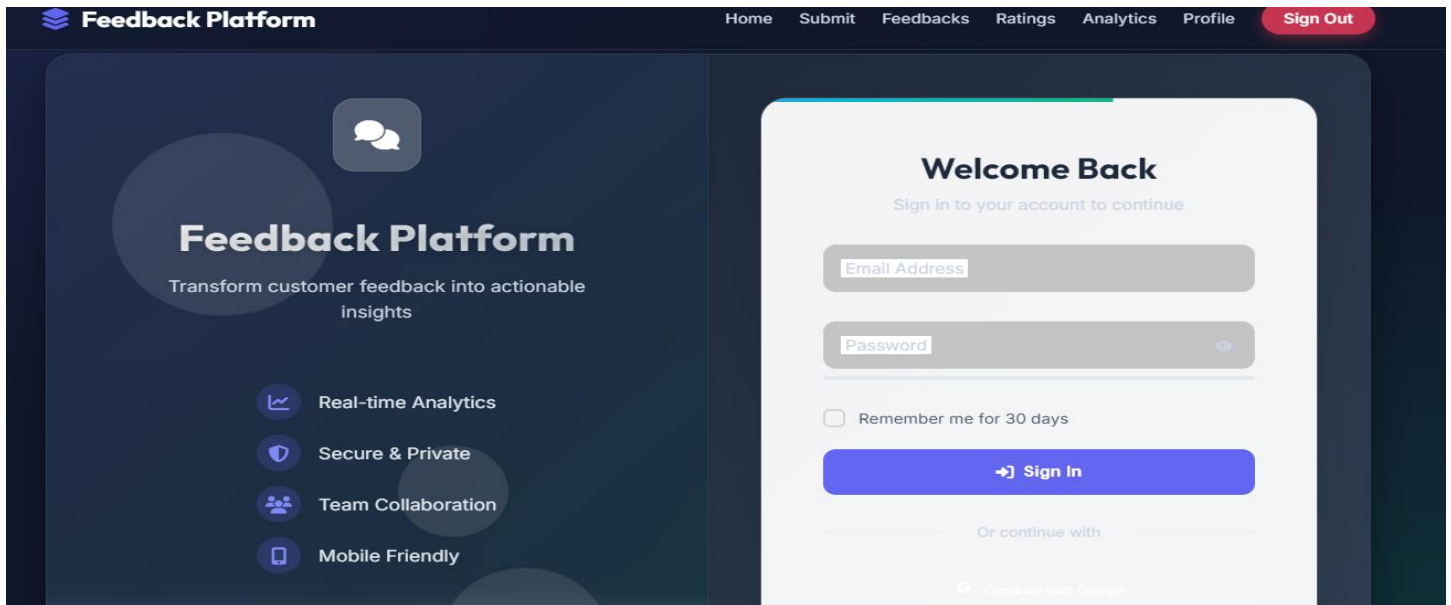


Figure 9:sign up page

8. Sign up

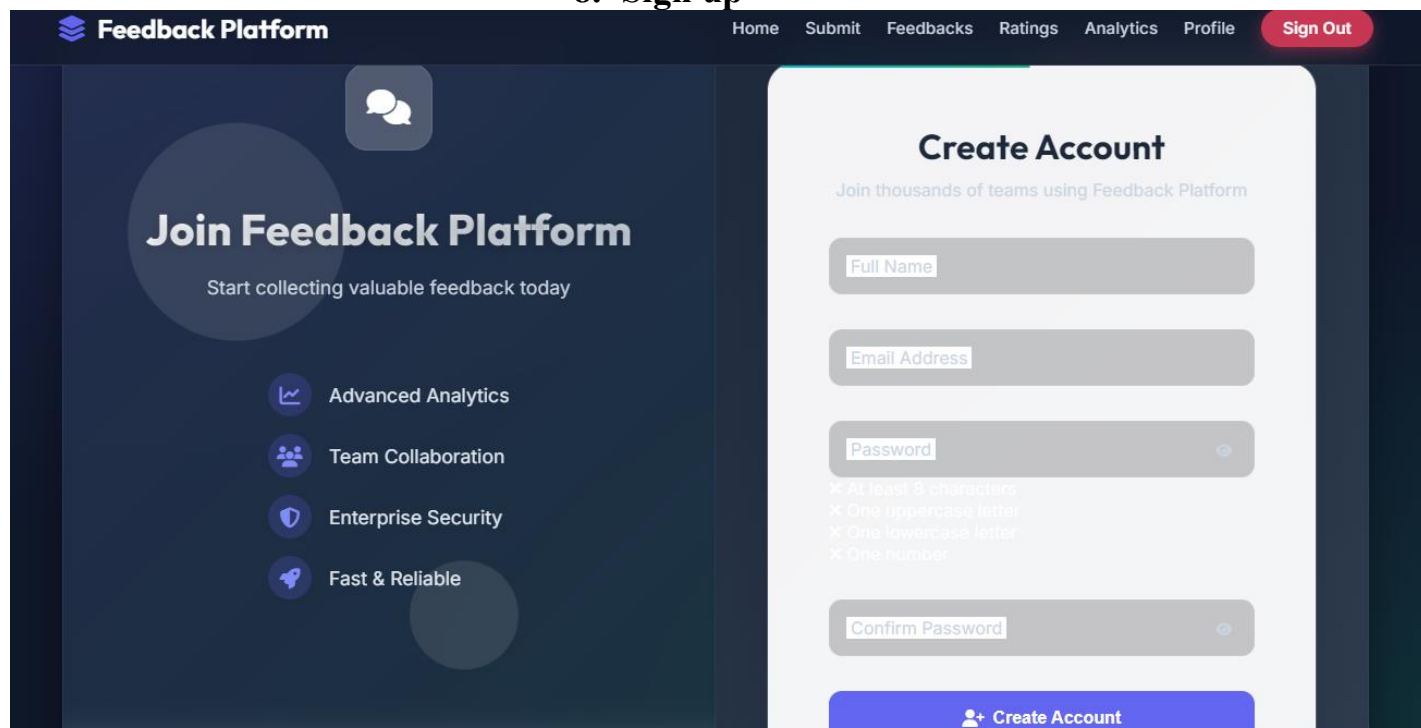


Figure 10: login page

23. CONCLUSION

The Feedback Platform represents a comprehensive solution to the feedback management challenges faced by educational institutions. By transitioning from manual, periodic processes to an automated, real-time system, the platform enables continuous improvement cycles that benefit all stakeholders. The implementation of modern web technologies (PHP, MySQL, responsive CSS) ensures reliability and accessibility, while the focus on user experience drives adoption and engagement.